

Predicting Insurance Premiums

Author: Negar Fathi

Student ID: 400222069

1. Introduction

In this project, I worked on predicting the Policy Cost for a fictional insurance company using a large anonymized dataset. We're talking over a million rows, with a good mix of numerical, categorical, and date-type features. The goal was to build a regression model that performs well under the RMSLE metric. Along the way, I explored the data, created new features, tried out different models, and looked into which features actually matter for predictions.

2. Data

2.1 Loading and Preview

The training set has about 1.1 million rows and 20 input features, including a mix of numerical, categorical, and temporal variables. The test set has the same structure, but without the target Policy Cost. The goal is to build a regression model that generalizes well to the test data.

2.2 EDA

The first thing I noticed was that the target variable was skewed to the right as seen in Figure 1 and the skewness was 1.24 which is not ideal for models like linear regression. We tried log1p transformation, but it flipped the skew too far to the left and the skewness became -1.26. To get a more balanced shape, we applied the Yeo-Johnson transformation. This method works on both positive and zero values and does not push the distribution too far in either direction. It made the target look close to normal. Skewness became -0.06, which is helpful for models like linear regression or SVR.

We also checked the distribution of 8 numeric features as seen in Figure 2. Years Lived, Dependent Count, Automobile Age, Financial Rating, and Coverage Period look uniform or flat. Yearly Earnings, Prior Claims, and Wellness Index are skewed to the right.

The boxplots in Figure 3 show that yearly earnings has many outliers.

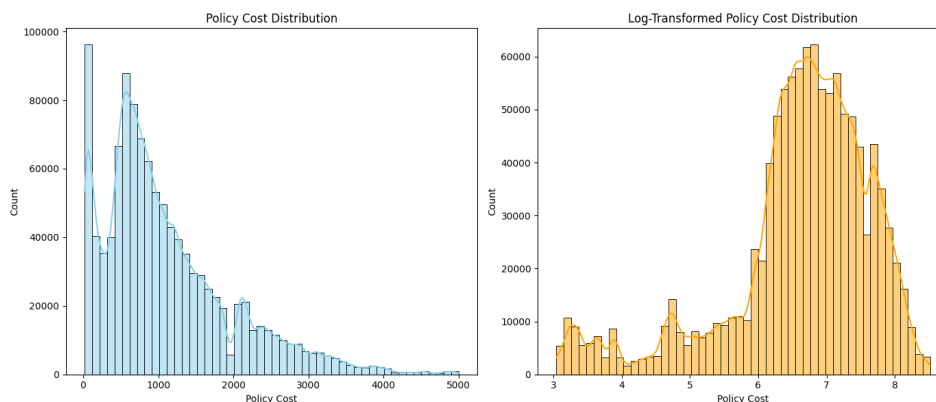


Figure 1

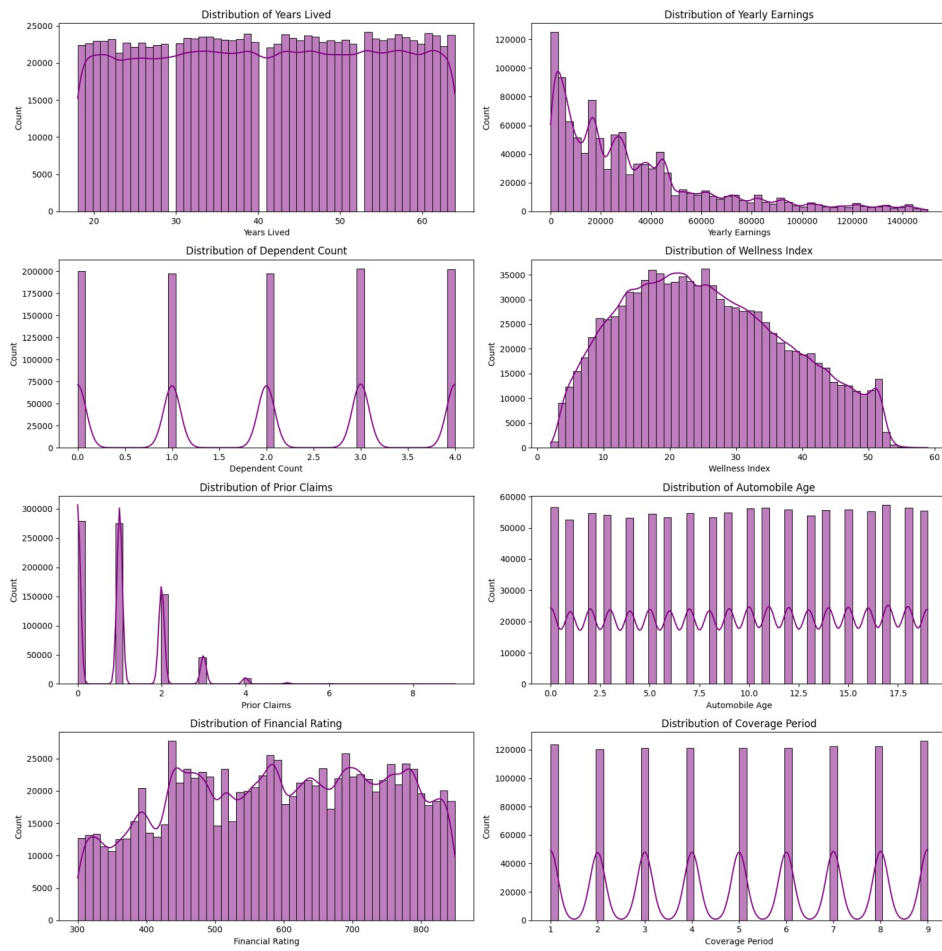


Figure 2

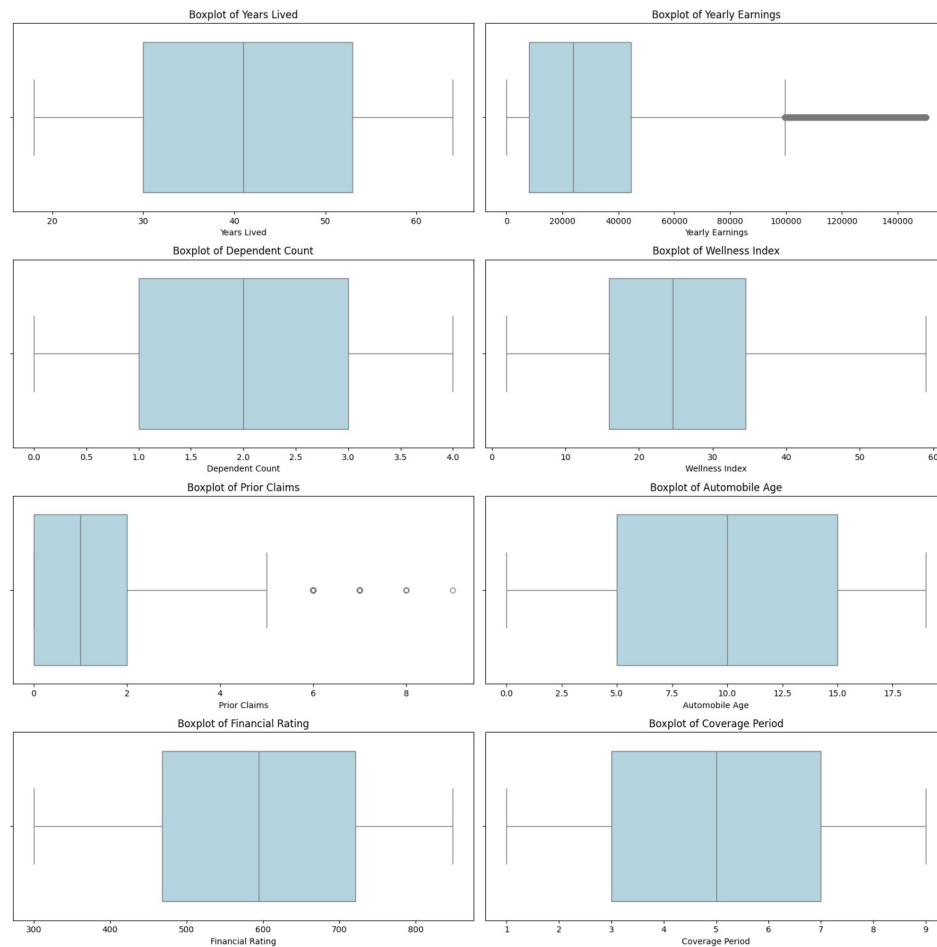


Figure 3

2.3 Handling Missing Values

To address missing data, we began by calculating the percentage of missing values for each column. The most affected columns were Prior Claims with 30.35% of data missing, Job Title with 29.87%, and Financial Rating with 11.49%. Based on the type and importance of each feature, we applied different imputation strategies.

For categorical columns such as Job Title, Client Review, and Relationship Status, we replaced missing values with the string "Missing" to retain the categorical structure while signaling the absence of information. This avoids discarding rows and allows the model to potentially learn patterns from missingness.

For numerical columns with moderate missingness, including Dependent Count, Yearly Earnings, Years Lived, Financial Rating, and Wellness Index, we used median imputation. Median is more robust than the mean in the presence of skewed distributions and outliers, which we had identified earlier during exploratory analysis.

The Prior Claims feature had the highest rate of missingness and is likely to be informative. To preserve its relationship with other features, we used Iterative Imputation, a multivariate technique that

models each feature with missing values as a function of other features in a round-robin fashion. This method can better estimate missing values when linear relationships exist among features.

Lastly, for Automobile Age and Coverage Period, which had a negligible number of missing values with less than 0.001% of data missing, we chose to drop the affected rows instead of imputing them, as this would not significantly affect the overall dataset size or distribution.

After applying these strategies, the dataset no longer contained missing values and was ready for further preprocessing.

2.4 Preprocessing

The preprocessing phase involved transforming categorical and date-based features into numerical representations suitable for modeling.

We first addressed ordinal categorical features. These have a natural order and were therefore mapped manually to preserve their ranking. For instance, Academic Standing was encoded from "Secondary" (0) up to "Doctorate" (3). Similarly, Physical Activity, Client Review, and Coverage Class were encoded based on logical progressions in activity, satisfaction, and policy coverage level, respectively.

The Coverage Commencement column was originally a datetime string. To make it machine-readable, we converted it into a proper datetime format and extracted informative components such as year, month, day, and day of the week. These features can help models identify seasonal or temporal patterns in policy cost. Once extracted, the original datetime column was dropped.

Next, for nominal categorical variables without inherent order including Sex, Relationship Status, Job Title, Region, Tobacco Use, and Asset Category we applied one-hot encoding using `pandas.get_dummies`. This method creates binary indicator columns for each unique category while retaining all values, as no single baseline category was assumed.

After these steps, all categorical and temporal data was numerically encoded, and the dataset was in a clean format, ready for model training.

3. Baseline Model Performance

We trained three baseline regression models and evaluated them using the Root Mean Squared Logarithmic Error (RMSLE), which is the competition metric.

The Linear Regression model scored an RMSLE of about 1.17. This simple model provides a baseline but struggles to capture non-linear relationships.

The Decision Tree Regressor, with pruning to avoid overfitting, performed better with an RMSLE around 1.13. It can model more complex patterns in the data.

The XGBoost model also achieved an RMSLE near 1.13, close to the Decision Tree but with slightly different strengths due to boosting.

Both tree-based models clearly outperform linear regression, suggesting non-linear interactions are important. We will build on these results with more feature engineering and tuning to improve performance further.

Model	RMSLE Score
Linear Regression	1.1698
Decision Tree	1.1319
XGBoost	1.1343

4. Feature Engineering

To improve the predictive power of our models, we applied a structured feature engineering process aimed at transforming and enhancing the original dataset into a model-ready form. This involved identifying and reducing redundancy, capturing interactions, and appropriately encoding categorical variables.

4.1 Identifying Redundant Features Using Cramer's V

We began by analyzing pairwise correlations between categorical features using *Cramer's V* which is a metric designed to measure the strength of association between two categorical variables. This approach is especially helpful when we can't use traditional correlation coefficients because data is non-numeric.

The heatmap based on *Cramer's V* values showed 0 associations across all pairs, indicating no strong multicollinearity between categorical variables. As a result, we decided to retain most of the base categorical features, avoiding premature elimination.

4.2 Creating Interaction Features

Next, to capture potential nonlinear relationships and dependencies between categorical variables, we created several interaction features by combining two or more columns:

- Sex_RelStatus = Sex + Relationship Status
- Region_Academic = Region + Academic Standing
- TobaccoUse_PhysicalActivity = Tobacco Use + Physical Activity
- Region_CoverageClass = Region + Coverage Class

The intuition here was that certain combinations of demographic and regional factors might provide more predictive signal than their individual parts.

To validate these engineered features, we computed their mutual information (MI) scores with the target variable Policy Cost. MI measures how much knowing one variable reduces uncertainty about another. It works for both linear and nonlinear relationships.

The results showed that interaction features contributed higher mutual information than their component features:

- Region_Academic: 0.0257 vs. Region: 0.0050 and Academic Standing: 0.0075

- Sex_RelStatus: 0.0165 vs. Sex: 0.0025 and Relationship Status: 0.0077
- Region_CoverageClass: 0.0189 vs. Region: 0.0050 and Coverage Class: 0.0049
- TobaccoUse_PhysicalActivity: 0.0166 vs. Tobacco Use: 0.0024 and Physical Activity: 0.0077

This confirmed that the engineered features captured more informative patterns than their individual variables.

4.3 Encoding and Transformation

Once feature selection and creation were completed, we proceeded with encoding similar to the preprocessing step. For features with a natural order, we used ordinal encoding. The Coverage Commencement column was converted to datetime, and useful subcomponents like Year, Month, Day, and Day of Week were extracted. The original datetime column was then dropped.

For nominal categorical features without any inherent order, we applied one-hot encoding using `pd.get_dummies`, avoiding the loss of information by retaining Sex, Relationship Status, Job Title, Region, Tobacco Use, Asset Category categories.

Interaction columns like Region_Academic and TobaccoUse_PhysicalActivity were label-encoded for simplicity, as they had too many combinations for one-hot encoding and did not imply an ordinal scale.

Finally, we dropped unused columns, such as the raw versions of encoded features, the original ID column, and certain one-hot columns based on mutual information analysis and to prevent overfitting.

4.4 Modeling

After applying feature engineering, we trained three baseline models on the transformed data:

Model	RMSLE Score
Linear Regression	1.1700
Decision Tree	1.1334
XGBoost	1.1430

These results showed little improvement in predictive performance compared to unprocessed data so it seems like feature engineering didn't improve the results that much. In the case of Linear Regression and Decision Tree it actually worsened the result.

5. Model Testing and Final Predictions

After training and validating our models, we applied our final pipeline to the test dataset. The following preprocessing steps were taken to ensure compatibility and consistency with the training data:

1. Missing Values Handling: Categorical columns (Job Title, Client Review, Relationship Status) were filled with 'Missing'. Numerical columns (e.g., Yearly Earnings, Wellness Index) were imputed using median values computed from the training data. Prior Claims was imputed using iterative imputation with the previously fitted imputer.

2. Feature Engineering and Encoding: Ordinal variables (Academic Standing, Physical Activity, Client Review, Coverage Class) were mapped manually based on logical order. Coverage Commencement was converted to datetime and decomposed into numerical features (year, month, day, weekday). One-hot encoding was applied to nominal categorical variables using `pd.get_dummies`, ensuring alignment with training features.
3. Feature Alignment: We explicitly reindexed the test set to match the training feature set, filling any missing columns with zero to maintain consistency.
4. Scaling: Numeric features were scaled using the standard scaler fitted during training.
5. Prediction: Decision Tree was used to generate predictions.
This model was chosen due to its lowest RMSLE score (1.1334) on validation compared to Linear Regression and XGBoost regressors.

6. Conclusion

Our analysis found that the most influential factors in predicting Policy Cost were financial Rating, Yearly Earnings, Wellness Index and Vehicle Age. These features had the strongest correlation with the target and were consistently selected by our model.

Surprisingly, categorical demographic variables such as Region, Sex, and Relationship Status showed little to no predictive power. Even engineered interaction features contributed minimally.

7. Business Recommendations

Features like Financial Rating and Wellness Index directly influence cost prediction. Efforts should be made to ensure these fields are complete, accurate, and consistently defined across clients.

We should also deprioritize low-impact demographic features in future modeling. Since features like Region and Sex showed negligible effect on policy cost, they can be excluded or deprioritized which reduces noise and streamlines model complexity.